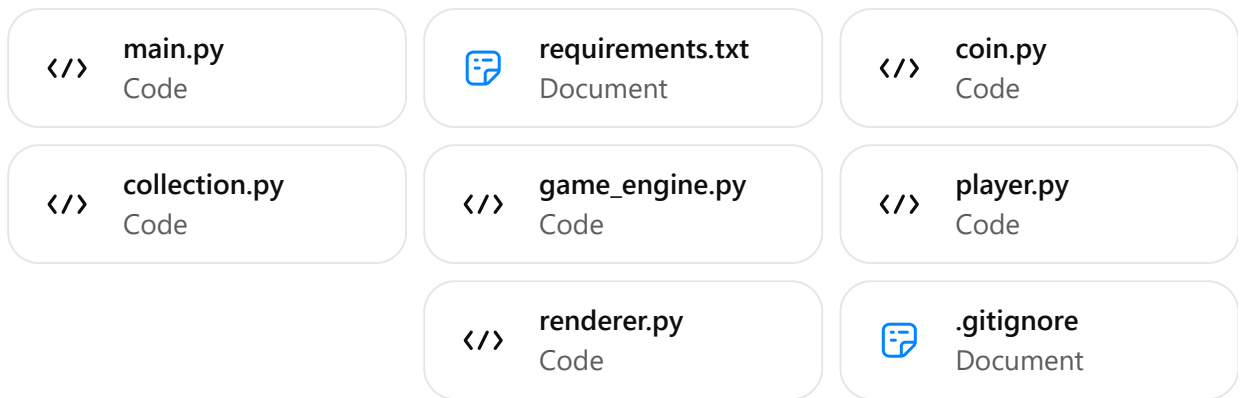


Today 12:16 PM



i have a lab assignment called coin collector, a small top down game written in python using pygame. i have attached the project files.

the project has a player controlled using the arrow keys, coins scattered around the play area, and a running score. but it has one bug and three additional feature tasks that i must add. i need to complete each of these tasks one after the other. for now work only on task 1. Do not implement any of the later features yet.

task 1 — fix the coin-collection bug-

a coin is supposed to be collected exactly once when the player touches it.

but now, `update()` in `game/game_engine.py` calls

`check_collection` every frame. as long as the player's rectangle overlaps a coin, its value keeps getting added to the score because the coin is never actually removed after collection. walking through one coin can therefore award its value many times.

the required behavior is:

when the player touches a coin, its value should be added to the score exactly once.

The collected coin should no longer be available after.

standing on the same position must not continue increasing the score.

do not implement multiple coin types, obstacles, lives, timers, or restart functionality yet.

Please first inspect the existing code I provide, especially

`game/game_engine.py` , `game/collection.py` , `game/coin.py` , and any other relevant files.

then:

explain why the current bug occurs, referring to the actual existing

code.

tell m the smallest appropriate change needed to fix it.

Tell me exactly which file need to change and show me the code changes.

explain the changes simply so that I can answer viva.

Tell me how to run and manually test the game to verify that a coin is collected exactly once.

Please do not unnecessarily rewrite or restructure working parts of the project. Base your solution on the code I upload rather than assuming what the files contain.

I inspected the uploaded code, and for **Task 1 only**, the fix is very small. You do **not** need to modify `collection.py` , `coin.py` , `player.py` , `renderer.py` , or `main.py` .

1. Why the bug occurs

Your main game loop calls:

Python

```
engine.update()
```

once every frame, and the game is capped at **60 frames per second**. main

Inside `GameEngine.update()` , the current code is:

Python

```
def update(self):
    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
```

game_engine

`check_collection()` checks every coin and returns all coins whose rectangles currently overlap the player's rectangle:

Python

```
for coin in coins:
    if player_rect.colliderect(coin.get_rect()):
        collected.append(coin)
```

</> collection

The collision detection itself is **working correctly**.

The problem is that after this happens:

</> Python

```
self.score += coin.value
```

the collected coin is **still inside** `self.coins` .

So imagine the player stands on a coin:

</> Plain text

```
Frame 1: collision → +1
Frame 2: same coin still exists → collision → +1
Frame 3: same coin still exists → collision → +1
Frame 4: same coin still exists → collision → +1
...
```

Since the game can run at 60 FPS, the score can increase extremely quickly while the player remains on one coin.

There's another visible consequence: `renderer.draw_scene()` draws **every coin that remains in** `self.coins` . </> renderer So because the collected coin isn't removed from the list, it also remains visible.

2. Smallest appropriate fix

Only change:

game/game_engine.py

After adding a collected coin's value to the score, remove that coin from `self.coins` .

Current code

</> Python

```
def update(self):
    collected = check_collection(self.player, self.coins)
```

```
for coin in collected:
    self.score += coin.value
```

Change it to

Python

```
def update(self):
    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
        self.coins.remove(coin)
```

So the **only new line** required is:

Python

```
self.coins.remove(coin)
```

No other files need to change.

This also fits the structure already provided by the starter code: `GameEngine` owns `self.coins`, while `check_collection()` simply determines which coins are currently being touched.

game_engine collection

3. Why this fix works

Suppose:

Python

```
self.coins = [coin1, coin2, coin3]
```

and the player touches `coin2`.

First:

Python

```
collected = check_collection(self.player, self.coins)
```

returns:

```
</> Python
```

```
[coin2]
```

Then:

```
</> Python
```

```
self.score += coin.value
```

adds `coin2` 's value **once**.

Finally:

```
</> Python
```

```
self.coins.remove(coin)
```

changes the game's coin list to:

```
</> Python
```

```
[coin1, coin3]
```

On the **next frame**, `check_collection()` receives only those remaining coins. `coin2` no longer exists in the active coin list, so it cannot be detected again and cannot award more points.

It also disappears visually automatically because the renderer loops through `self.coins` when drawing. `</> renderer`

Important implementation detail

We're **not** modifying `self.coins` while directly iterating through `self.coins` .

We're iterating through:

```
</> Python
```

```
for coin in collected:
```

where `collected` is a separate list returned by `check_collection()` .

Therefore removing a coin from `self.coins` here is appropriate.

4. Simple viva explanation

A good viva answer would be:

" `update()` runs once every frame. `check_collection()` correctly detected when the player overlapped a coin, but the collected coin was never removed from `self.coins`. So on every following frame, the same coin was detected again and its value was repeatedly added to the score. I fixed it by removing each collected coin from `self.coins` immediately after adding its value. Therefore it cannot be detected or drawn in later frames."

If they ask **why you didn't change `check_collection()`**, you can say:

" `check_collection()` has one responsibility: detecting which coins currently collide with the player. The `GameEngine` owns the coin list and score, so updating the game state by removing a collected coin belongs in `GameEngine.update()`."

That's a nice separation-of-responsibilities answer.

5. Exactly what you need to edit

Open:

 Plain text

`game/game_engine.py`

Find:

 Python

```
def update(self):
    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
```

Add one line:

 Python

```
def update(self):
    collected = check_collection(self.player, self.coins)
    for coin in collected:
```

```
self.score += coin.value  
self.coins.remove(coin)
```

Task 1 is complete with that change.

6. How to run and manually test it

Your `main.py` specifically says the project runs with `python3 main.py`, and the controls are the arrow keys. `</>` `main` The only dependency listed is Pygame 2.5 or newer.  `requirements`

From the project's root directory, install the dependency if necessary:

`</>` **Bash**

```
pip install -r requirements.txt
```

Then run:

`</>` **Bash**

```
python main.py
```

or, depending on your system:

`</>` **Bash**

```
python3 main.py
```

Use the **arrow keys** to move the blue player onto a coin.

Check three things:

1. **Score:** touching one coin should increase the score from `0` to `1` exactly once. The starter currently gives every generated coin `COIN_VALUE = 1`. `</>` `game_engine`
2. **Removal:** the coin should immediately disappear.
3. **No repeated scoring:** stop moving and remain where the coin was. The score should stay at `1`, rather than continuing `2, 3, 4, 5...`

Then move onto another coin. The score should become `2`, and that coin should disappear too.

That directly verifies the requirement: **one collision** → **one score addition** → **coin removed** → **impossible to collect again**.

And importantly, this solution changes **only Task 1 behavior**—it does not introduce coin types, obstacles, lives, timers, or restarting.

i am on command prompt with a local repo
C:\Users\tanus\06_coin_collector\coin-collector>c
how do i make the changes in this and commit it because each
task needs a commit separatly

Yes — since **each task needs its own commit**, do only the one-line Task 1 fix now, test it, then commit it before touching Task 2.

You're already in:

</> Plain text

```
C:\Users\tanus\06_coin_collector\coin-collector>
```

1. Open `game_engine.py`

If you use VS Code, from Command Prompt run:

</> bat

```
code game\game_engine.py
```

If `code` isn't recognized, you can use Notepad:

</> bat

```
notepad game\game_engine.py
```

Find:

</> Python

```
def update(self):  
    collected = check_collection(self.player, self.coins)  
    for coin in collected:  
        self.score += coin.value
```

Add the one line:

</> Python

```
def update(self):
    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
        self.coins.remove(coin)
```

Save the file.

2. Run and test Task 1

Back in Command Prompt:

```
</> bat
python main.py
```

Move onto a coin with the arrow keys. Confirm that the coin **disappears**, the score increases by **exactly 1**, and standing where the coin was does **not** keep increasing the score.

Close the game after testing.

3. Check exactly what Git sees

Run:

```
</> bat
git status
```

You should see something like:

```
</> Plain text
modified:   game/game_engine.py
```

Then inspect your change:

```
</> bat
git diff
```

Ideally, the meaningful diff should just be:

```
</> Diff
```

```
def update(self):
    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
+       self.coins.remove(coin)
```

This is a good habit for your lab because it proves you haven't accidentally included Task 2 work in the Task 1 commit.

4. Stage only the Task 1 file

```
</> bat
git add game/game_engine.py
```

Then check:

```
</> bat
git status
```

game/game_engine.py should now appear under **Changes to be committed**.

5. Make the Task 1 commit

I'd use a clear commit message:

```
</> bat
git commit -m "Fix coin collection to remove collected coins"
```

Or, if your instructor wants the task explicitly identified:

```
</> bat
git commit -m "Task 1: Fix repeated coin collection bug"
```

I prefer the second one for a lab assignment because your Git history will be very easy to demonstrate during evaluation.

6. Verify the commit

Run:

```
</> bat
git log --oneline
```

You should see something similar to:

```
</> Plain text
a1b2c3d Task 1: Fix repeated coin collection bug
...
```

At that point, **stop before making Task 2 changes**. Your repository now has a clean checkpoint containing only the Task 1 solution.

So your workflow for every task will essentially be:

```
</> Plain text
make Task 1 change
  ↓
test it
  ↓
git diff
  ↓
git add ...
  ↓
git commit
  ↓
start Task 2
```

This will give you a separate, easy-to-explain commit for every requirement.

now move on to the second task-
add 3 coin types: bronze (1 point), silver (3 points), and gold (5 points), with different colors. make sure each gives the correct score when collected. only make the changes needed for task 2 along with the codes and where i need to paste it and don't implement tasks 3 or 4 yet. explain what you changed and how i can test it.

Since Task 1 already made `GameEngine` remove collected coins, Task 2 can stay nicely contained. Based on your starter code, you only need to change `game/game_engine.py` and `game/coin.py`. We won't touch obstacles, lives, timers, or restart functionality.

1. Change `game/coin.py`

Your `Coin` class already supports both a `value` and a `color` :

`</>` Python

```
def __init__(self, x, y, radius=12, value=1, color=(230, 190, 60)):
```

`</>` coin

So we don't need to redesign the class. We just need a convenient definition of the three coin types.

Add this ABOVE `class Coin`:

Open:

`</>` Plain text

```
game\coin.py
```

After:

`</>` Python

```
import pygame
```

paste:

`</>` Python

```
COIN_TYPES = {
    "bronze": {"value": 1, "color": (205, 127, 50)},
    "silver": {"value": 3, "color": (192, 192, 192)},
    "gold": {"value": 5, "color": (255, 215, 0)},
}
```

So the beginning of the file becomes:

`</>` Python

```
import pygame
```

```
COIN_TYPES = {
    "bronze": {"value": 1, "color": (205, 127, 50)},
    "silver": {"value": 3, "color": (192, 192, 192)},
    "gold": {"value": 5, "color": (255, 215, 0)},
}
```

```
class Coin:
    def __init__(self, x, y, radius=12, value=1, color=(230, 190, 60)):
        self.x = x
        self.y = y
        self.radius = radius
        self.value = value
        self.color = color
```

That's all you need to change in `coin.py` .

2. Change `game/game_engine.py`

Currently the starter imports:

`</>` Python

```
from game.coin import Coin
```

`</>` game_engine

Change it to:

`</>` Python

```
from game.coin import Coin, COIN_TYPES
```

Then remove this line

Your existing file has:

`</>` Python

```
COIN_VALUE = 1
```

`</>` game_engine

Delete it because coins no longer all have the same value.

Keep:

`</>` Python

NUM_COINS = 6

3. Change `_random_coin()`

Your current method is:

`</>` Python

```
def _random_coin(self):
    x = random.randint(30, WIDTH - 30)
    y = random.randint(30, HEIGHT - 30)
    return Coin(x=x, y=y, radius=12, value=COIN_VALUE)
```

`</>` game_engine

Replace that whole method with:

`</>` Python

```
def _random_coin(self):
    x = random.randint(30, WIDTH - 30)
    y = random.randint(30, HEIGHT - 30)

    coin_type = random.choice(list(COIN_TYPES.values()))

    return Coin(
        x=x,
        y=y,
        radius=12,
        value=coin_type["value"],
        color=coin_type["color"]
    )
```

That's the main Task 2 change.

Your Task 1 `update()` should **stay exactly as you changed it**:

`</>` Python

```
def update(self):
    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
        self.coins.remove(coin)
```

Notice why Task 2 already works with Task 1: the score uses `coin.value` , rather than adding a hardcoded `1` . `</>` `game_engine`

What did we actually change?

Previously every coin was created with:

`</>` Python

```
value=COIN_VALUE
```

and:

`</>` Python

```
COIN_VALUE = 1
```

So every coin was worth exactly one point.

Now we define three possibilities:

Coin	Value	Colour
Bronze	1	Bronze/brown
Silver	3	Silver/grey
Gold	5	Gold/yellow

Whenever `_random_coin()` creates a coin, this line:

`</>` Python

```
coin_type = random.choice(list(COIN_TYPES.values()))
```

randomly selects one of those three configurations.

For example, it might select:

`</>` Python

```
{"value": 5, "color": (255, 215, 0)}
```

Then the created `Coin` gets:

`</>` Python

```
value=5
color=(255, 215, 0)
```

The renderer already uses:

`</>` Python

```
coin.color
```

when drawing each coin, so **no change to `renderer.py` is necessary.** `</>` `renderer`

And your collection code already adds:

`</>` Python

```
self.score += coin.value
```

so **no new scoring logic is necessary either.**

That's why this is a small change rather than restructuring the project.

Simple viva explanation

You can say:

"The existing `Coin` class already had `value` and `color` attributes, so I reused them instead of creating separate classes for bronze, silver and gold. I defined the three coin types with their corresponding values and RGB colors. When a coin is generated, the game randomly chooses one of these types and passes its value and color to the existing `Coin` constructor. The existing collection logic already adds `coin.value`, so bronze adds 1, silver adds 3, and gold adds 5."

If they ask why you didn't create `BronzeCoin`, `SilverCoin`, and `GoldCoin` classes:

"The only differences between the three coins are their value and color, so separate classes would add unnecessary complexity. The existing `Coin` class already supports both properties."

How to test Task 2

Run:

```
</> bat
```

```
python main.py
```

You should now see the six coins in a mixture of **bronze, silver, and gold**. Because the types are randomly selected, you might occasionally not get all three in one run; just restart the game if necessary.

Then collect them one at a time while watching the score:

```
</> Plain text
```

```
Bronze → score increases by 1
```

```
Silver → score increases by 3
```

```
Gold   → score increases by 5
```

For example, if you collect:

```
</> Plain text
```

```
Bronze → 0 + 1 = 1
```

```
Gold   → 1 + 5 = 6
```

```
Silver → 6 + 3 = 9
```

Each coin should also disappear after being collected because of your Task 1 fix.

Then make your separate Task 2 commit

Before committing, check:

```
</> bat
```

```
git diff
```

You should see Task 2 changes in `game/coin.py` and `game/game_engine.py`.

Then:

```
</> bat
```

```
git add game/coin.py game/game_engine.py
```

```
git commit -m "Task 2: Add bronze silver and gold coin types"
```

Finally:

`</> bat`

`git log --oneline`

You should now have something like:

`</> Plain text`

```
xxxxxxx Task 2: Add bronze silver and gold coin types
yyyyyyy Task 1: Fix repeated coin collection bug
6217e10 Initial Coin Collector starter
```

That gives you a clean Task 1 commit and a separate Task 2 commit, with **nothing from Tasks 3 or 4 included**.

```
C:\Users\tanus\06_coin_collector\coin-collector>python main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.11.9)
Hello from the pygame community.
https://www.pygame.org/contribute.html
Traceback (most recent call last):
  File "C:\Users\tanus\06_coin_collector\coin-collector\main.py",
line 11, in <module>
    from game.game_engine import GameEngine
  File "C:\Users\tanus\06_coin_collector\coin-
collector\game\game_engine.py", line 28
    x = random.randint(30, WIDTH - 30)
    ^
IndentationError: expected an indented block after function
definition on line 27
```

```
C:\Users\tanus\06_coin_collector\coin-collector>
this happened when i tried to run the game
```

Yep, this is just an **indentation error**, not a problem with the Task 2 logic.

Python is telling you:

`</> Plain text`

```
IndentationError: expected an indented block after function definition on line 27
```

That means your file probably currently looks like this:

</> Python

```
def _random_coin(self):
    x = random.randint(30, WIDTH - 30)
    y = random.randint(30, HEIGHT - 30)
```

Everything inside a function must be indented.

Fix game\game_engine.py

Make sure your `_random_coin()` section looks **exactly** like this:

</> Python

```
def _random_coin(self):
    x = random.randint(30, WIDTH - 30)
    y = random.randint(30, HEIGHT - 30)

    coin_type = random.choice(list(COIN_TYPES.values()))

    return Coin(
        x=x,
        y=y,
        radius=12,
        value=coin_type["value"],
        color=coin_type["color"]
    )
```

Pay attention to the indentation:

</> Plain text

```
class GameEngine:
|
| 4 spaces → def _random_coin(self):
|
|   8 spaces → x = ...
|   8 spaces → y = ...
|   8 spaces → coin_type = ...
|
|   8 spaces → return Coin(
|       12 spaces → x=x,
|       12 spaces → y=y,
|       ...
```

Your surrounding section should therefore look like:

</> Python

```

class GameEngine:
    def __init__(self):
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
        self.coins = [self._random_coin() for _ in range(NUM_COINS)]
        self.score = 0

    def _random_coin(self):
        x = random.randint(30, WIDTH - 30)
        y = random.randint(30, HEIGHT - 30)

        coin_type = random.choice(list(COIN_TYPES.values()))

        return Coin(
            x=x,
            y=y,
            radius=12,
            value=coin_type["value"],
            color=coin_type["color"]
        )

```

Save it and run again:

</> bat

python main.py

Don't commit Task 2 yet. First make sure the game opens, you see the different-colored coins, and bronze/silver/gold give +1 , +3 , and +5 respectively. Then we'll commit it separately.

Today 9:31 PM

```

C:\Users\tanus\06_coin_collector\coin-collector>git diff
diff --git a/coin-collector/game/coin.py b/coin-collector/game/coin.py
index c4f7533..43df2d2 100644
--- a/coin-collector/game/coin.py
+++ b/coin-collector/game/coin.py
@@ -5,6 +5,11 @@ bounding square around it.

```

import pygame

```

+COIN_TYPES = {
+    "bronze": {"value": 1, "color": (205, 127, 50)},

```

```
+ "silver": {"value": 3, "color": (192, 192, 192)},
+ "gold": {"value": 5, "color": (255, 215, 0)},
+ }
```

```
class Coin:
    def __init__(self, x, y, radius=12, value=1, color=(230, 190, 60)):
```

```
diff --git a/coin-collector/game/game_engine.py b/coin-collector/game/game_engine.py
```

index 63293ae..e3f7e88 100644

```
--- a/coin-collector/game/game_engine.py
+++ b/coin-collector/game/game_engine.py
@@ -11,12 +11,11 @@ import random
import pygame
```

```
from game.player import Player
-from game.coin import Coin
+from game.coin import Coin, COIN_TYPES
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT
```

```
NUM_COINS = 6
-COIN_VALUE = 1
```

```
class GameEngine:
    @@ -26,9 +25,18 @@ class GameEngine:
        self.score = 0
```

```
def _random_coin(self):  
-     x = random.randint(30, WIDTH - 30)  
-     y = random.randint(30, HEIGHT - 30)  
-     return Coin(x=x, y=y, radius=12, value=COIN_VALUE)  
+     x = random.randint(30, WIDTH - 30)  
+     y = random.randint(30, HEIGHT - 30)  
+  
.
```

```
diff --git a/coin-collector/game/coin.py b/coin-  
collector/game/coin.py
```

index c4f7533..43df2d2 100644

```
--- a/coin-collector/game/coin.py
```

```
+++ b/coin-collector/game/coin.py
```

```
@@ -5,6 +5,11 @@ bounding square around it.
```

```
import pygame
```

```
+COIN_TYPES = {
+    "bronze": {"value": 1, "color": (205, 127, 50)},
+    "silver": {"value": 3, "color": (192, 192, 192)},
+    "gold": {"value": 5, "color": (255, 215, 0)},
+}
```

```
class Coin:
```

```
    def __init__(self, x, y, radius=12, value=1, color=(230, 190, 60)):
```

```
diff --git a/coin-collector/game/game_engine.py b/coin-
collector/game/game_engine.py
```

```
index 63293ae..e3f7e88 100644
```

```
--- a/coin-collector/game/game_engine.py
```

```
+++ b/coin-collector/game/game_engine.py
```

```
@@ -11,12 +11,11 @@ import random
```

```
import pygame
```

```
from game.player import Player
```

```
-from game.coin import Coin
```

```
+from game.coin import Coin, COIN_TYPES
```

```
from game.collection import check_collection
```

```
from game.renderer import WIDTH, HEIGHT
```

```
NUM_COINS = 6
```

```
-COIN_VALUE = 1
```

```
class GameEngine:
```

```
@@ -26,9 +25,18 @@ class GameEngine:
```

```
    self.score = 0
```

```
    def _random_coin(self):
```

```
-    x = random.randint(30, WIDTH - 30)
```

```
-    y = random.randint(30, HEIGHT - 30)
```

```
-    return Coin(x=x, y=y, radius=12, value=COIN_VALUE)
```

```
+    x = random.randint(30, WIDTH - 30)
```

```
+     y = random.randint(30, HEIGHT - 30)
+
+
:
```

what to do now

You're almost there. The `git diff` output is being shown through Git's pager, which is why you see the `:` at the bottom and the diff appears repeated.

First: exit `git diff`

Press:

`</>` Plain text

q

That should return you to:

`</>` Plain text

```
C:\Users\tanus\06_coin_collector\coin-collector>
```

Then make sure the game actually works

Your diff shows the Task 2 changes we wanted: `COIN_TYPES` was added, `COIN_VALUE` was removed, and `GameEngine` now imports `COIN_TYPES`.

However, I notice the diff displays only **7 spaces** before these lines:

`</>` Python

```
x = random.randint(30, WIDTH - 30)
y = random.randint(30, HEIGHT - 30)
```

If `python main.py` now runs successfully, don't worry about that display. If you still get an `IndentationError`, make sure `_random_coin()` is exactly:

`</>` Python

```
def _random_coin(self):
    x = random.randint(30, WIDTH - 30)
    y = random.randint(30, HEIGHT - 30)

    coin_type = random.choice(list(COIN_TYPES.values()))
```

```
    return Coin(  
        x=x,  
        y=y,  
        radius=12,  
        value=coin_type["value"],  
        color=coin_type["color"]  
    )
```

Then run:

```
</> bat
```

```
python main.py
```

Verify that bronze gives +1, silver gives +3, gold gives +5, and each coin disappears after collection.

Once that works, commit Task 2

Run these commands **one at a time**:

```
</> bat
```

```
git status
```

Then:

```
</> bat
```

```
git add game/coin.py game/game_engine.py
```

Then:

```
</> bat
```

```
git status
```

You should see both files under **Changes to be committed**.

Then commit:

```
</> bat
```

```
git commit -m "Task 2: Add bronze silver and gold coin types"
```

Finally:

```
</> bat

git log --oneline
```

You should now have separate commits resembling:

```
</> Plain text

xxxxxxx Task 2: Add bronze silver and gold coin types
yyyyyyy Task 1: Fix repeated coin collection bug
6217e10 Initial Coin Collector starter
```

Don't start Task 3 until you see that Task 2 commit in `git log`. That keeps each assignment task cleanly separated.

now help me with task 3 implement obstacles. add obstacles that the player has to avoid, and make the player lose one out of 5 total lifes when they hit one. make sure one collision doesn't keep reducing lives every frame and that the obstacles stay inside the play area. only make the changes needed for task 3 and don't implement task 4 yet. explain what you changed and how i can test it.

Yes. We can add Task 3 on top of your Task 1 + Task 2 code without touching Task 4.

For this task, I'd make **three small changes**:

1. Add a simple `Obstacle` class.
2. Let `GameEngine` create obstacles, track 5 lives, and detect obstacle collisions.
3. Update the existing renderer so obstacles and lives are visible.

The important part is preventing one obstacle collision from taking a life **every frame**.

1. Create `game\obstacle.py`

Inside your `game` folder, create a new file:

```
</> Plain text

game\obstacle.py
```

Paste:

`</>` Python

```
import pygame

class Obstacle:
    def __init__(self, x, y, width=50, height=50):
        self.x = x
        self.y = y
        self.width = width
        self.height = height

    def get_rect(self):
        return pygame.Rect(
            self.x,
            self.y,
            self.width,
            self.height
        )
```

This is deliberately simple.

Just like `Player` and `Coin` have `get_rect()` methods for collision detection, the obstacle now has one too. Your existing player already exposes its rectangle through `get_rect()` . `</>` player

2. Modify `game\game_engine.py`

A. Add the import

At the top you currently have imports like:

`</>` Python

```
from game.player import Player
from game.coin import Coin, COIN_TYPES
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT
```

Add:

`</>` Python

```
from game.obstacle import Obstacle
```

So it becomes:

`</>` Python

```
from game.player import Player
from game.coin import Coin, COIN_TYPES
from game.obstacle import Obstacle
from game.collection import check_collection
from game.renderer import WIDTH, HEIGHT
```

B. Add number of obstacles

Under:

`</>` Python

```
NUM_COINS = 6
```

add:

`</>` Python

```
NUM_OBSTACLES = 4
```

So:

`</>` Python

```
NUM_COINS = 6
NUM_OBSTACLES = 4
```

We're using four obstacles simply to give the player something meaningful to avoid.

3. Add lives and obstacles in `__init__`

Your current `__init__` should have:

`</>` Python

```
def __init__(self):
    self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
```

```
self.coins = [self._random_coin() for _ in range(NUM_COINS)]
self.score = 0
```

Change it to:

`</>` Python

```
def __init__(self):
    self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
    self.coins = [self._random_coin() for _ in range(NUM_COINS)]
    self.obstacles = [self._random_obstacle() for _ in range(NUM_OBSTACLES)]
    self.score = 0
    self.lives = 5
    self.touching_obstacle = False
```

What are the two new state variables?

`</>` Python

```
self.lives = 5
```

The player starts with the required **5 lives**.

And:

`</>` Python

```
self.touching_obstacle = False
```

This is important. It remembers whether the player was already touching an obstacle on the previous frame.

Without this, we'd create the exact same kind of problem you had with Task 1:

`</>` Plain text

```
frame 1 → collision → lose life
frame 2 → collision → lose life
frame 3 → collision → lose life
...
```

Instead, we're going to lose a life only when the player **first enters** an obstacle.

4. Add `_random_obstacle()`

Put this method after `_random_coin()` :

`</>` Python

```
def _random_obstacle(self):
    width = 50
    height = 50

    x = random.randint(0, WIDTH - width)
    y = random.randint(0, HEIGHT - height)

    return Obstacle(x, y, width, height)
```

Make sure its indentation matches `_random_coin()` .

Why `WIDTH - width` ?

This fulfills your requirement that obstacles stay inside the play area.

The renderer defines the game area as `700 × 500` . `</>` `renderer`

If an obstacle is 50 pixels wide:

`</>` Plain text

```
maximum x = WIDTH - 50
```

Therefore its right edge can never extend past the window.

Same for:

`</>` Python

```
HEIGHT - height
```

on the vertical axis.

5. Add obstacle collision logic to `update()`

After Task 1, your current `update()` should be:

`</>` Python

```
def update(self):
    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
        self.coins.remove(coin)
```

Keep all of that.

Add the obstacle logic underneath it:

 Python

```
def update(self):
    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
        self.coins.remove(coin)

    player_rect = self.player.get_rect()
    hitting_obstacle = any(
        player_rect.colliderect(obstacle.get_rect())
        for obstacle in self.obstacles
    )

    if hitting_obstacle and not self.touching_obstacle:
        self.lives -= 1

    self.touching_obstacle = hitting_obstacle
```

This is the most important Task 3 code.

How does it prevent repeated life loss?

Suppose you walk into an obstacle.

Initially:

 Plain text

```
touching_obstacle = False
```

You enter it:

 Plain text

```
hitting_obstacle = True
touching_obstacle = False
```

Therefore:

</> Python

```
if hitting_obstacle and not self.touching_obstacle:
```

is:

</> Plain text

```
True AND True
```

So:

</> Python

```
self.lives -= 1
```

You go from:

</> Plain text

```
5 → 4
```

Then we store:

</> Python

```
self.touching_obstacle = hitting_obstacle
```

so it becomes:

</> Plain text

```
touching_obstacle = True
```

Next frame

You're still standing on the obstacle:

</> Plain text

```
hitting_obstacle = True
touching_obstacle = True
```

The condition becomes:

```
</> Plain text
```

```
True AND NOT True
```

which is false.

So you **don't lose another life**.

Once you leave

```
</> Plain text
```

```
hitting_obstacle = False
```

so:

```
</> Plain text
```

```
touching_obstacle = False
```

If you walk into an obstacle again later, the condition becomes true again and you lose another life.

That's exactly the behaviour we want.

6. Pass obstacles to the renderer

At the bottom of `game_engine.py`, your current `draw()` is based on:

```
</> Python
```

```
renderer.draw_scene(surface, self.player, self.coins)
renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
```

```
</> game_engine
```

Change it to:

Python

```
def draw(self, surface, font):  
    from game import renderer  
    renderer.draw_scene(surface, self.player, self.coins, self.obstacles)  
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))  
    renderer.draw_text(surface, font, f"Lives: {self.lives}", (10, 40))
```

This displays the five lives underneath the score.

7. Modify `game\renderer.py`

We need the obstacles to actually appear.

Your renderer currently has:

Python

```
COLOR_BG = (35, 45, 35)  
COLOR_PLAYER = (80, 180, 255)  
COLOR_TEXT = (255, 255, 255)
```

renderer

Add:

Python

```
COLOR_OBSTACLE = (180, 60, 60)
```

So:

Python

```
COLOR_BG = (35, 45, 35)  
COLOR_PLAYER = (80, 180, 255)  
COLOR_TEXT = (255, 255, 255)  
COLOR_OBSTACLE = (180, 60, 60)
```

Then your existing `draw_scene()` is:

Python

```
def draw_scene(surface, player, coins):
    surface.fill(COLOR_BG)
    for coin in coins:
        pygame.draw.circle(surface, coin.color, (int(coin.x), int(coin.y)), coin.radius)
    pygame.draw.rect(surface, COLOR_PLAYER, player.get_rect(), border_radius=4)
```

⌄ render

Change it to:

⌄ Python

```
def draw_scene(surface, player, coins, obstacles):
    surface.fill(COLOR_BG)

    for coin in coins:
        pygame.draw.circle(
            surface,
            coin.color,
            (int(coin.x), int(coin.y)),
            coin.radius
        )

    for obstacle in obstacles:
        pygame.draw.rect(
            surface,
            COLOR_OBSACLE,
            obstacle.get_rect()
        )

    pygame.draw.rect(
        surface,
```

Nothing fancy — obstacles are simply red rectangles.

Files changed for Task 3

You should now have:

⌄ Plain text

```
game\obstacle.py      ← NEW
game\game_engine.py   ← MODIFIED
game\renderer.py      ← MODIFIED
```

Don't modify `coin.py`, `collection.py`, `player.py`, or `main.py` for this task.

Viva explanation

You can explain Task 3 like this:

"I added an Obstacle class which stores its position and dimensions and provides a rectangle for collision detection. The game engine randomly creates obstacles while restricting their coordinates so the entire obstacle remains inside the game window. The player starts with five lives."

Then the important part:

"Since update runs every frame, simply subtracting a life whenever a collision exists would remove many lives from one collision. So I store whether the player was already touching an obstacle. A life is deducted only when the current frame has a collision but the previous frame did not. The player has to leave the obstacle before another collision can cost another life."

If asked about this:

`</>` Python

```
any(  
    player_rect.collidect(obstacle.get_rect())  
    for obstacle in self.obstacles  
)
```

say:

" `collidect` checks whether the player's rectangle overlaps an obstacle's rectangle. `any` becomes true if the player is colliding with at least one obstacle."

Test it before committing

Run:

`</>` bat

```
python main.py
```

You should initially see:

`</>` Plain text

Score: 0

Lives: 5

along with your bronze/silver/gold coins and four red obstacles.

Walk into an obstacle.

You should get:

</> Plain text

Lives: 4

Stop moving while you're still touching it.

The lives must remain:

</> Plain text

Lives: 4

and must NOT rapidly become:

</> Plain text

Lives: 3

Lives: 2

Lives: 1

Lives: 0

...

Then walk completely away from the obstacle and hit it again:

</> Plain text

Lives: 3

Also check that all four obstacles are completely inside the window.

Finally, make sure Task 2 still works: bronze +1 , silver +3 , gold +5 , and collected coins disappear.

Then check the diff

</> bat

```
git diff
```

You should see modifications to `game_engine.py` and `renderer.py` .

Because `obstacle.py` is a brand-new untracked file, also run:

```
</> bat
```

```
git status
```

You should see something like:

```
</> Plain text
```

```
modified:   game/game_engine.py
```

```
modified:   game/renderer.py
```

```
untracked:  game/obstacle.py
```

Then commit **only Task 3**:

```
</> bat
```

```
git add game/game_engine.py game/renderer.py game/obstacle.py
```

Check:

```
</> bat
```

```
git status
```

Then:

```
</> bat
```

```
git commit -m "Task 3: Add obstacles and player lives"
```

And verify:

```
</> bat
```

```
git log --oneline
```

You should now have separate Task 1, Task 2, and Task 3 commits. **Don't add any game-over/restart functionality when lives reach zero yet**—that would start creeping into Task 4.

the logic of task 3 is not complete. the game should end after all 5 lives are lost and also i want to make the obstacle glow when player hits it to make the lives being lost evident. i want the lives to not just be a count down but actually have 5 red hearts and upon hitting a obstacle one of the hearts loses red and becomes an outline only. help me implement all this in the easiest simplest way possible, give me the code to paste and tell me where to do it

Yes — with your clarification, Task 3 should include a proper **game-over state**, clear collision feedback, and visual lives. We can do all three without making the code complicated.

Keep your existing `obstacle.py` . We only need to adjust `game_engine.py` and `renderer.py` .

1. `game_engine.py` — track the hit obstacle and game over

A. In `__init__`

You currently added:

`</>` Python

```
self.lives = 5
self.touching_obstacle = False
```

Add two more variables directly below them:

`</>` Python

```
self.hit_obstacle = None
self.game_over = False
```

So that part becomes:

`</>` Python

```
self.score = 0
self.lives = 5
self.touching_obstacle = False
self.hit_obstacle = None
self.game_over = False
```

`hit_obstacle` tells the renderer **which obstacle should glow**.

`game_over` tells the rest of the game whether all lives have been lost.

B. Stop movement after game over

At the very beginning of your existing `handle_input()` method, add:

`</>` Python

```
if self.game_over:
    return
```

So it starts like:

`</>` Python

```
def handle_input(self, keys_pressed):
    if self.game_over:
        return

    dx = dy = 0
```

This means once the player loses all five lives, arrow-key movement stops.

C. Replace your Task 3 collision section in `update()`

Keep your coin collection code.

Make your entire `update()` :

`</>` Python

```
def update(self):
    if self.game_over:
        return

    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
        self.coins.remove(coin)

    player_rect = self.player.get_rect()
```

```

hit_obstacle = None
for obstacle in self.obstacles:
    if player_rect.colliderect(obstacle.get_rect()):
        hit_obstacle = obstacle
        break

hitting_obstacle = hit_obstacle is not None

```

What's different?

Previously we only knew:

`</>` Python

```
hitting_obstacle = True
```

But now we actually save the obstacle:

`</>` Python

```
hit_obstacle = obstacle
```

and then:

`</>` Python

```
self.hit_obstacle = hit_obstacle
```

That lets us tell the renderer:

This specific obstacle is currently being hit — make this one glow.

When you leave:

`</>` Python

```
if not hitting_obstacle:
    self.hit_obstacle = None
```

so the glow disappears.

And when:

`</>` Python

```
self.lives == 0
```

we set:

`</>` Python

```
self.game_over = True
```

2. Change draw() in game_engine.py

Replace your current draw() with:

`</>` Python

```
def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.player,
        self.coins,
        self.obstacles,
        self.hit_obstacle
    )

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_hearts(surface, self.lives)
```

Notice we are **removing**:

`</>` Python

```
renderer.draw_text(surface, font, f"Lives: {self.lives}", (10, 40))
```

because the hearts now represent the lives.

Your starter renderer already has `draw_banner()` , so we can reuse it rather than inventing another game-over rendering system. `</>` `renderer`

3. `renderer.py` — make the obstacle glow

You previously added:

`</>` Python

```
COLOR_OBSACLE = (180, 60, 60)
```

Under it, add:

`</>` Python

```
COLOR_OBSACLE_HIT = (255, 255, 120)
COLOR_HEART = (220, 50, 50)
```

The yellow-ish hit colour will make the obstacle visibly flash/glow while the player is touching it.

4. Replace `draw_scene()`

Change:

`</>` Python

```
def draw_scene(surface, player, coins, obstacles):
```

to:

`</>` Python

```
def draw_scene(surface, player, coins, obstacles, hit_obstacle):
```

Then replace the obstacle-drawing loop with this:

`</>` Python

```
for obstacle in obstacles:
    if obstacle is hit_obstacle:
        pygame.draw.rect(
```

```

        surface,
        COLOR_OBSACLE_HIT,
        obstacle.get_rect(),
        border_radius=6
    )

    pygame.draw.rect(
        surface,
        COLOR_OBSACLE_HIT,
        obstacle.get_rect().inflate(10, 10),
        width=3,
        border_radius=8
    )
else:
    pygame.draw.rect(
        surface,
        COLOR_OBSACLE,

```

So a normal obstacle is red.

When hit, it becomes bright and gets a larger outline around it:

</> Plain text



It's a simple way of making the collision visually obvious without adding animations or timers.

5. Add the hearts to `renderer.py`

Now add this new function **below** `draw_text()` :

</> Python

```

def draw_hearts(surface, lives):
    start_x = 15
    y = 55
    spacing = 35

    for i in range(5):
        x = start_x + i * spacing

```

```

points = [
    (x, y + 5),
    (x - 8, y - 3),
    (x - 15, y + 3),
    (x - 14, y + 11),
    (x, y + 25),
    (x + 14, y + 11),
    (x + 15, y + 3),
    (x + 8, y - 3),
]

if 5 < 10:

```

This avoids needing heart images or external assets.

At the start:

</> Plain text

♥ ♥ ♥ ♥ ♥

After one hit:

</> Plain text

♥ ♥ ♥ ♥ ♥

After another:

</> Plain text

♥ ♥ ♥ ♥ ♥

and eventually:

</> Plain text

♥ ♥ ♥ ♥ ♥

The actual display will be drawn heart shapes rather than text characters.

How the heart code works

This loop always draws **five** hearts:

</> Python

```
for i in range(5):
```

If that heart represents a remaining life:

</> Python

```
if i < lives:
```

we use:

</> Python

```
pygame.draw.polygon(surface, COLOR_HEART, points)
```

which fills it red.

Otherwise:

</> Python

```
pygame.draw.polygon(  
    surface,  
    COLOR_HEART,  
    points,  
    width=2  
)
```

The `width=2` means **draw only the outline**.

So when:

</> Python

```
lives = 3
```

the indexes behave like:

</> Plain text

```
i = 0    filled  
i = 1    filled  
i = 2    filled
```

```
i = 3    outline  
i = 4    outline
```

Very simple logic for your viva.

What happens during a collision now?

Suppose you have:

```
</> Plain text
```

♥ ♥ ♥ ♥ ♥

You hit an obstacle.

Three things happen together:

1. One life is removed

```
</> Python
```

```
self.lives -= 1
```

giving:

```
</> Plain text
```

♥ ♥ ♥ ♥ ♥

2. The obstacle is remembered

```
</> Python
```

```
self.hit_obstacle = hit_obstacle
```

so the renderer makes that obstacle glow.

3. `touching_obstacle` becomes `True`

so remaining on that obstacle does **not** deduct another life.

Once you move away, the glow disappears and `touching_obstacle` returns to `False`.

Then you're able to lose another life from the next collision.

What happens on the fifth hit?

Suppose:

</> Plain text

```
Lives = 1
```

You hit an obstacle:

</> Python

```
self.lives -= 1
```

Now:

</> Plain text

```
Lives = 0
```

so:

</> Python

```
if self.lives == 0:  
    self.game_over = True
```

becomes true.

The next input/update is stopped by:

</> Python

```
if self.game_over:  
    return
```

and `draw()` displays:

</> Plain text

GAME OVER

using the `draw_banner()` function your starter project already provided. `</>` `renderer`

We are **not adding restart functionality**. Once the game ends, the user simply closes it. That keeps us within Task 3 and out of Task 4.

Test everything before committing

Run:

```
</> bat
```

```
python main.py
```

Start with five filled hearts. Hit an obstacle once: one heart should become an outline and the obstacle should glow.

Stay touching the obstacle for several seconds. You must remain at four lives.

Move completely away. The obstacle should stop glowing.

Hit it again. Another heart should empty.

Repeat until the fifth collision. You should reach five outlined hearts, see **GAME OVER**, and arrow keys should no longer move the player.

Also quickly collect a bronze/silver/gold coin to make sure Tasks 1 and 2 still work.

Then:

```
</> bat
```

```
git diff
```

and:

```
</> bat
```

```
git status
```

If everything looks right, stage your Task 3 files:

`</> bat`

```
git add game/game_engine.py game/renderer.py game/obstacle.py
```

Then commit:

`</> bat`

```
git commit -m "Task 3: Add obstacles lives and game over"
```

That will keep this entire completed obstacle/lives functionality together as your **Task 3 commit**, without introducing Task 4.

now help me complete task 4- implement a timed round. add a 30 second countdown and display the remaining time clearly on the screen. the round should end when either the timer reaches 0 or the player loses all 5 lives. when the round ends, stop normal gameplay, clearly show the final score, and give the player a simple way to start a new round. restarting should reset the score, lives, timer, coins, obstacles, and player properly. keep all the task 1, 2, and 3 functionality working and only make the changes needed for task 4. give me the simplest implementation possible, tell me exactly what code to add or change and where, and explain how i can test all the task 4 requirements.

Task 4 can be added cleanly on top of Tasks 1–3. The simplest approach is to use Pygame's `pygame.time.get_ticks()` to measure the 30-second round, then use **R** to restart after the round ends.

You only need to modify `game/game_engine.py`. Your existing `renderer.py` already has `draw_text()` and `draw_banner()`, so we can reuse them. `</> renderer`

1. Add the round duration

Open:

`</> Plain text`

```
game\game_engine.py
```

Near the top you currently have:

`</> Python`

```
NUM_COINS = 6
NUM_OBSTACLES = 4
```

Add:

`</>` Python

```
ROUND_TIME = 30
```

So:

`</>` Python

```
NUM_COINS = 6
NUM_OBSTACLES = 4
ROUND_TIME = 30
```

This makes the round duration easy to understand and change.

2. Simplify `__init__` by using a reset method

Your current `__init__()` contains all the setup for the player, coins, obstacles, score, lives, etc.

For restarting, we need to perform **exactly that setup again**.

Replace your current `__init__()` with:

`</>` Python

```
def __init__(self):
    self.reset()
```

Then directly underneath it add:

`</>` Python

```
def reset(self):
    self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
    self.coins = [self._random_coin() for _ in range(NUM_COINS)]
    self.obstacles = [self._random_obstacle() for _ in range(NUM_OBSTACLES)]

    self.score = 0
    self.lives = 5
```

```

self.touching_obstacle = False
self.hit_obstacle = None
self.game_over = False

self.start_time = pygame.time.get_ticks()

```

So the beginning of your class will look roughly like:

Python

```

class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
        self.coins = [self._random_coin() for _ in range(NUM_COINS)]
        self.obstacles = [self._random_obstacle() for _ in range(NUM_OBSTACLES)]

        self.score = 0
        self.lives = 5

        self.touching_obstacle = False
        self.hit_obstacle = None
        self.game_over = False

        self.start_time = pygame.time.get_ticks()

    def _random_coin(self):
        ...

```

Why do this?

Previously the setup only happened when `GameEngine` was first created.

Now:

Python

```

self.reset()

```

can perform the same setup whenever the player starts a new round.

That automatically resets:

- player position
- coins
- obstacles

- score
- 5 lives
- collision state
- game-over state
- timer

So we don't need separate reset logic for every variable.

3. Add a method to calculate the remaining time

After `_random_obstacle()` , add:

`</>` Python

```
def get_time_left(self):
    elapsed = (pygame.time.get_ticks() - self.start_time) // 1000
    return max(0, ROUND_TIME - elapsed)
```

How this works

Pygame:

`</>` Python

```
pygame.time.get_ticks()
```

returns the number of milliseconds since Pygame started.

We saved the round's starting time:

`</>` Python

```
self.start_time = pygame.time.get_ticks()
```

Then:

`</>` Python

```
pygame.time.get_ticks() - self.start_time
```

tells us how long the current round has been running.

We divide by 1000:

Python

```
// 1000
```

to convert milliseconds into whole seconds.

Then:

Python

```
ROUND_TIME - elapsed
```

gives the countdown.

So you get:

Plain text

```
30
29
28
27
...
2
1
0
```

`max(0, ...)` prevents it displaying negative numbers.

4. Change `handle_input()` to allow restart

You currently have this from Task 3:

Python

```
def handle_input(self, keys_pressed):
    if self.game_over:
        return

    dx = dy = 0
```

Change that beginning to:

</> Python

```
def handle_input(self, keys_pressed):
    if self.game_over:
        if keys_pressed[pygame.K_r]:
            self.reset()
        return

    dx = dy = 0
```

Leave the rest of your movement code exactly as it is.

So when the round is over:

- arrow keys don't move the player
- pressing R calls `reset()`
- a completely fresh round starts

5. Add the timer check to `update()`

Your Task 3 `update()` begins:

</> Python

```
def update(self):
    if self.game_over:
        return
```

Immediately after that, add:

</> Python

```
if self.get_time_left() == 0:
    self.game_over = True
    return
```

So the beginning becomes:

</> Python

```
def update(self):
    if self.game_over:
        return

    if self.get_time_left() == 0:
```

```

        self.game_over = True
        return

    collected = check_collection(self.player, self.coins)

```

Leave all your existing coin and obstacle collision code underneath it unchanged.

Your Task 3 code already handles the other ending condition:

`</>` Python

```

if self.lives == 0:
    self.game_over = True

```

Therefore there are now **two ways to reach game over**:

`</>` Plain text

```

timer reaches 0
    OR
lives reach 0
    ↓
game_over = True

```

6. Change `draw()` to display the timer and final score

Keep your existing scene drawing and hearts.

Make your `draw()` method:

`</>` Python

```

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.player,
        self.coins,
        self.obstacles,
        self.hit_obstacle
    )

    renderer.draw_text(
        surface,

```

```

        font,
        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_hearts(surface, self.lives)

```

That's all we need for the UI.

During normal gameplay you'll have something like:

`</>` Plain text

Score: 8 Time: 24

♥ ♥ ♥ ♥ ♥

At the end:

`</>` Plain text

GAME OVER - Final Score: 8 - Press R to restart

Your existing `draw_banner()` centers the supplied text on the screen. `</>` `renderer`

Your final Task 4 logic

Conceptually it's now very simple:

`</>` Plain text

Start round

↓

reset()

↓

score = 0

lives = 5

timer = 30

new coins

new obstacles

player centered

↓

PLAY

↓



```

timer = 0?      lives = 0?
|               |
└──────────┬───┘
            |
            ↓
        GAME OVER

```

Viva: explain the timer

A good short answer:

"I used `pygame.time.get_ticks()` to record when the round starts. During the game, I subtract the starting time from the current Pygame time and convert milliseconds to seconds. I subtract that elapsed time from 30 to get the remaining countdown."

If they ask why you used:

Python

```
max(0, ROUND_TIME - elapsed)
```

say:

"The max prevents the displayed timer from becoming negative after the round ends."

Viva: explain restarting

"I moved all round initialization into a `reset()` method. Both the constructor and the restart functionality call the same method. This avoids duplicating reset code and ensures the score, lives, timer, player, coins, obstacles and collision state are all reset consistently."

That's probably the most important design decision in Task 4.

Test 1 — Countdown

Run:

</> bat

```
python main.py
```

Check that you initially see approximately:

</> Plain text

Time: 30

It should count:

</> Plain text

30
29
28
...

without you doing anything.

Test 2 — Timer reaches zero

For testing, you don't need to sit through 30 seconds repeatedly.

Temporarily change:

</> Python

```
ROUND_TIME = 30
```

to:

</> Python

```
ROUND_TIME = 5
```

Run the game.

Check:

</> Plain text

5
4
3
2
1
0

At zero:

- game ends
- player can't move normally
- final score appears
- restart instruction appears

Then change it back to 30 before committing.

Test 3 — Game ends from lives

Start another round.

Hit an obstacle, move away, hit again, and repeat.

Your hearts should continue working:

</> Plain text



On the fifth collision, the game should immediately end even if there is still time remaining.

That verifies:

</> Plain text

timer reaches 0 OR lives reach 0

Test 4 — Final score

Collect some coins before ending.

For example:

</> Plain text

bronze +1
silver +3

```
gold    +5
```

```
score = 9
```

Then let the timer expire.

The end message should show:

```
</> Plain text
```

```
Final Score: 9
```

and the score should no longer change.

Test 5 — Restart

Once game over appears, press:

```
</> Plain text
```

```
R
```

Verify **all** of these:

- score returns to 0
- lives return to five full hearts
- timer returns to 30
- player returns to the center
- new coins appear
- new obstacles appear
- old collision/glow state is gone
- player can move again

This is the most important Task 4 test because the assignment explicitly requires everything to reset properly.

Test 6 — Make sure Tasks 1–3 still work

In the restarted round:

Task 1: collect one coin and stand there → score only increases once.

Task 2: bronze +1 , silver +3 , gold +5 .

Task 3: hit obstacle → exactly one heart empties and the obstacle glows; staying on it doesn't continuously remove lives.

Task 4: timer counts down, either zero lives or zero time ends the game, final score appears, R starts a fresh round.

Then commit Task 4 separately

First:

```
</> bat
```

```
git diff
```

For Task 4, you should mainly see changes to:

```
</> Plain text
```

```
game/game_engine.py
```

Then:

```
</> bat
```

```
git status
```

If everything is correct:

```
</> bat
```

```
git add game/game_engine.py
```

Then:

```
</> bat
```

```
git commit -m "Task 4: Add timed rounds and restart"
```

Finally:

```
</> bat
```

```
git log --oneline
```

You should now have a very clean assignment history:

```
</> Plain text
```

```
xxxxxxx Task 4: Add timed rounds and restart
xxxxxxx Task 3: Add obstacles lives and game over
xxxxxxx Task 2: Add bronze silver and gold coin types
xxxxxxx Task 1: Fix repeated coin collection bug
xxxxxxx Initial Coin Collector starter
```

That clean history will also make it much easier to explain exactly what you implemented for each task during your viva.

there are some implementation mistakes-

- the coins should keep generating for example if i claim one coin another one should pop up in a random location
- the timer should pause after all lives are lost so that when game ends player can see how much time was left

Yes — both should be corrected. You **do not need to restructure Task 4**. Make two small changes in `game/game_engine.py` .

1. Generate a replacement coin whenever one is collected

Your current Task 1 collection section should look like:

```
</> Python
```

```
collected = check_collection(self.player, self.coins)
for coin in collected:
    self.score += coin.value
    self.coins.remove(coin)
```

Change it to:

```
</> Python
```

```
collected = check_collection(self.player, self.coins)
for coin in collected:
    self.score += coin.value
```

```
self.coins.remove(coin)
self.coins.append(self._random_coin())
```

That's it.

What happens now

If there are initially 6 coins:

 Plain text

```
6 coins
↓
player collects gold
↓
+5 score
↓
gold coin removed
↓
new random coin created
↓
still 6 coins
```

Because `_random_coin()` already randomly selects bronze/silver/gold and a random position, the replacement automatically gets a random type and location.

For viva:

"After a coin is collected, I remove it so it can't be collected twice, then immediately append a newly generated random coin. This keeps the number of active coins constant."

2. Freeze the timer when the player loses all lives

Right now `get_time_left()` calculates the time from the current Pygame ticks:

 Python

```
def get_time_left(self):
    elapsed = (pygame.time.get_ticks() - self.start_time) // 1000
    return max(0, ROUND_TIME - elapsed)
```

The problem is that even after:

 Python

```
self.game_over = True
```

`pygame.time.get_ticks()` keeps increasing.

So the displayed countdown keeps decreasing.

The simplest fix is to **store the remaining time when the game ends.**

A. In `reset()`

Find:

 Python

```
self.game_over = False
```

```
self.start_time = pygame.time.get_ticks()
```

Add:

 Python

```
self.final_time = None
```

So:

 Python

```
self.game_over = False
```

```
self.final_time = None
```

```
self.start_time = pygame.time.get_ticks()
```

3. Modify `get_time_left()`

Replace your current method:

 Python

```
def get_time_left(self):  
    elapsed = (pygame.time.get_ticks() - self.start_time) // 1000  
    return max(0, ROUND_TIME - elapsed)
```

with:

 Python

```
def get_time_left(self):
    if self.final_time is not None:
        return self.final_time

    elapsed = (pygame.time.get_ticks() - self.start_time) // 1000
    return max(0, ROUND_TIME - elapsed)
```

Now, once we save `final_time`, the method simply returns that frozen value.

4. Freeze the time when the fifth life is lost

In your obstacle collision code you should currently have:

 Python

```
if hitting_obstacle and not self.touching_obstacle:
    self.lives -= 1
    self.hit_obstacle = hit_obstacle

if self.lives == 0:
    self.game_over = True
```

Change only the final part to:

 Python

```
if hitting_obstacle and not self.touching_obstacle:
    self.lives -= 1
    self.hit_obstacle = hit_obstacle

if self.lives == 0:
    self.final_time = self.get_time_left()
    self.game_over = True
```

The order matters.

Do:

 Python

```
self.final_time = self.get_time_left()
```

first, then:

</> Python

```
self.game_over = True
```

For example, if the fifth life is lost with 17 seconds remaining:

</> Plain text

```
final_time = 17  
game_over = True
```

After that, `get_time_left()` always returns:

</> Plain text

```
17
```

even if the player sits on the game-over screen for another minute.

What about when the timer itself reaches zero?

You already have:

</> Python

```
if self.get_time_left() == 0:  
    self.game_over = True  
    return
```

That's fine.

If the timer causes game over, the displayed time naturally stays at:

</> Plain text

```
Time: 0
```

So we only need to explicitly freeze it when **losing all lives ends the round early**.

Your relevant `update()` should now look like this

Don't blindly replace unrelated code, but compare your method against this:

`</>` Python

```
def update(self):
    if self.game_over:
        return

    if self.get_time_left() == 0:
        self.game_over = True
        return

    collected = check_collection(self.player, self.coins)
    for coin in collected:
        self.score += coin.value
        self.coins.remove(coin)
        self.coins.append(self._random_coin())

    player_rect = self.player.get_rect()

    hit_obstacle = None
    for obstacle in self.obstacles:
        if player_rect.colliderect(obstacle.get_rect()):
            hit_obstacle = obstacle
```

Test 1 — continuous coins

Run:

`</>` bat

```
python main.py
```

Count the coins. You should have 6.

Collect one.

Check that:

- its correct value is added to the score;
- the collected coin disappears;
- **a new coin immediately appears somewhere else;**
- there are still 6 coins.

Collect several more. You should never run out of coins.

Also make sure the replacement coins can still be bronze, silver, or gold.

Test 2 — frozen timer after losing lives

For easier testing, you can temporarily set:

</> Python

```
ROUND_TIME = 60
```

so you have plenty of time.

Hit an obstacle five separate times, moving away between hits.

Suppose your fifth heart disappears when:

</> Plain text

```
Time: 43
```

You should see game over with:

</> Plain text

```
Time: 43
```

Wait 5–10 seconds.

It should **still say**:

</> Plain text

```
Time: 43
```

That's the important test.

Then change the round time back to:

</> Python

ROUND_TIME = 30

before committing.

Test 3 — timer ending still works

Start another round and don't lose all your lives.

Let:

</> Plain text

Time: 3

Time: 2

Time: 1

Time: 0

At 0, game over should occur normally.

So you now have the two correct endings:

</> Plain text

Lose all 5 lives

→ GAME OVER

→ timer freezes at remaining time

Timer reaches 0

→ GAME OVER

→ timer stays at 0

Test 4 — restart resets these changes too

Press R after game over.

Make sure:

</> Plain text

Score → 0

Lives → 5 filled hearts

Time → 30

Coins → fresh set

```
Obstacles → fresh set
Player → starting position
Game → playable again
```

Then collect a coin and make sure a replacement still appears.

One small edge case to be aware of

Your `_random_coin()` currently chooses a completely random position. That means a newly generated coin **could occasionally appear on top of the player or an obstacle**. Your assignment only says random location, so you don't need to complicate the code unless the task specifically requires coins not to overlap obstacles/player.

For now, keeping `_random_coin()` simple is the better choice.

help me commit this

Since these fixes are part of **Task 4**, put them in the same Task 4 commit if you haven't committed Task 4 yet.

From:

```
</> Plain text
```

```
C:\Users\tanus\06_coin_collector\coin-collector>
```

First check what changed:

```
</> bat
```

```
git status
```

Then inspect it:

```
</> bat
```

```
git diff
```

Press `q` to exit the diff view.

If everything is working, stage `game_engine.py` :

```
</> bat
```

```
git add game/game_engine.py
```

Check:

```
</> bat
```

```
git status
```

You should see `game/game_engine.py` under **Changes to be committed**.

Then commit:

```
</> bat
```

```
git commit -m "Task 4: Add timed rounds and restart"
```

Finally verify:

```
</> bat
```

```
git log --oneline
```

You should now have something like:

```
</> Plain text
```

```
xxxxxxx Task 4: Add timed rounds and restart
xxxxxxx Task 3: Add obstacles lives and game over
xxxxxxx Task 2: Add bronze silver and gold coin types
xxxxxxx Task 1: Fix repeated coin collection bug
xxxxxxx Initial Coin Collector starter
```

If you **already committed Task 4 before making these two fixes**, don't run the command above yet—tell me, because then we'll make a small follow-up commit rather than accidentally mixing up your commit history.